UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026).
   Answers are unofficial reference answers, not official UTM marking schemes.

2. Marking points show the expected answer and mark allocation; check them against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Approved real questions only; no generators, placeholders or parametric variants.
   Questions and answers are for local study use only.

5. Structure derived from the supplied CSC429 papers; every question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 11 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION 1 (5 marks)

Source: CSC429-2015-06, page [3]

Unofficial model answer:

The complete machine cycle consists of four phases: Fetch, Decode, Execute, and Store (or Write-back). A common diagram is a cycle: Fetch → Decode → Execute → Store → (back to Fetch). Each phase is described below with its function and location.

Fetch: The CPU fetches the next instruction from main memory. The Program Counter (PC) holds the address of the next instruction; this address is placed on the address bus, the instruction is read from memory via the data bus, and then the PC is incremented to point to the next instruction. The fetched instruction is placed in the Instruction Register (IR). Location: This phase occurs in the CPU (Control Unit and registers) and involves a read from main memory (RAM).

Decode: The Control Unit decodes the instruction held in the IR to determine the operation to be performed, the addressing mode, and the operands. It generates the necessary control signals for the subsequent phases. Location: This phase occurs entirely within the CPU, specifically in the Control Unit and associated decoding logic.

Execute: The CPU performs the operation specified by the instruction. This may involve an arithmetic or logic operation in the ALU, a data transfer, or a branch. Operands may be read from registers or memory. Location: This phase occurs in the CPU (ALU and registers); if memory operands are required, it also involves main memory access.

Store (Write-back): The result of the execution is written back to a register or to main memory. For a memory write, the address and data are placed on the buses and the memory is updated. Location: This phase occurs in the CPU (register write) and/or main memory (memory write).

Note: The exact diagram may vary slightly (e.g., some texts include an interrupt check or combine store with execute), but the four-phase description above is standard.

Marking points:

• *1 mark: Correctly drawing the complete machine cycle showing the four phases (Fetch, Decode, Execute, Store/Write-back) in the correct sequence.*

• *1 mark: Describing the Fetch phase: fetching instruction from memory using PC, placing in IR, incrementing PC; location: CPU and main memory.*

• *1 mark: Describing the Decode phase: control unit decodes instruction in IR to determine operation and operands; location: CPU (Control Unit).*

• *1 mark: Describing the Execute phase: ALU performs operation or data transfer; location: CPU (ALU/registers) and possibly memory.*

• *1 mark: Describing the Store/Write-back phase: writing result back to register or memory; location: CPU registers and/or main memory.*

Model confidence: high.

Model note: The question asks for a drawing; the answer describes the diagram in text. The exact phases and labels may vary by textbook, but the four-phase model is standard. No figure was provided, so the answer is based on general computer organization knowledge.

QUESTION 2 (5 marks)

Source: CSC429-2018-01, page [3]

Unofficial model answer:

a) The zero generation (pre-1940s) machines are called mechanical computers because their computation and control were performed by physical moving parts—gears, levers, wheels, and shafts—rather than by electronic circuits. They had no electronic components, no stored-program memory, and no electronic switching; operations were driven by mechanical linkages and, in some cases, by external power such as hand cranks or steam. Hence 'mechanical' describes the implementation technology of the computing hardware.

b) Table completion:
- Mechanical Calculating Machine — Inventor: Blaise Pascal (1642); Function: performs arithmetic addition and subtraction (with carry) using gear wheels; a mechanical calculator for basic arithmetic.
- Difference Machine — Inventor: Charles Babbage (1822); Function: automatically computes and tabulates polynomial functions using the method of finite differences, producing printed mathematical tables without manual arithmetic errors.

Marking points:

• 1 mark: zero generation machines used mechanical moving parts (gears/levers/wheels) for computation and control.

• 1 mark: no electronics / no stored-program concept; operation by mechanical linkages (and possibly external power).

• 1 mark: Mechanical Calculating Machine inventor = Blaise Pascal.

• 1 mark: Mechanical Calculating Machine function = mechanical arithmetic (addition/subtraction) using gears.

• 1 mark: Difference Machine inventor = Charles Babbage and function = compute/tabulate polynomials by finite differences (automated mathematical tables).

Model confidence: high.

Model note: The original table layout is not supplied; the answer assumes the two rows are 'Mechanical Calculating Machine' and 'Difference Machine' and that 'Inventor' and 'Function' are the two columns. Some syllabi may accept Leibniz for the calculating machine; Pascal's Pascaline is the standard zero-generation example. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 3 (5 marks)

Source: CSC429-2015-06, page [2]

Unofficial model answer:

a) The Step Reckoner (also known as the Stepped Reckoner) was invented by Gottfried Wilhelm Leibniz. The Mechanical Calculating Machine (often referring to the Pascaline or the Arithmetical Machine) was invented by Blaise Pascal.

b) The main functional difference is that Pascal's machine could only perform addition and subtraction (with some sources noting it could also do multiplication and division through repeated operations, but it was limited to these arithmetic operations). Leibniz's Step Reckoner was designed to perform all four basic arithmetic operations: addition, subtraction, multiplication, and division directly. Additionally, the Step Reckoner used a stepped drum mechanism (Leibniz wheel) that allowed it to handle larger numbers and automate multiplication and division more efficiently.

c) The Step Reckoner (Leibniz's machine) is the one that uses the binary numbering system, which is fundamental in modern computers' operation. Leibniz was a proponent of the binary system and designed his machine to incorporate binary concepts, although the physical implementation used decimal digits with binary-coded positions.

Marking points:

• 1 mark: Correctly names Gottfried Wilhelm Leibniz as inventor of the Step Reckoner.

• 1 mark: Correctly names Blaise Pascal as inventor of the Mechanical Calculating Machine (Pascaline).

• 1 mark: Describes that Pascal's machine performed only addition and subtraction (or limited operations).

• 1 mark: Describes that Leibniz's Step Reckoner performed all four arithmetic operations (addition, subtraction, multiplication, division).

© Hak Cipta Universiti Teknologi MARA

• 1 mark: Identifies the Step Reckoner (Leibniz's machine) as using the binary numbering system.

Model confidence: medium.

Model note: The question statement mentions 'Zero generation' and 'mechanical calculating machine' without specifying the inventor; it is assumed to refer to Pascal's Pascaline, as is common in CSC429 history of computing. The Step Reckoner's use of binary is often cited in textbooks, though the machine itself operated on decimal digits; the binary reference is to Leibniz's conceptual contribution. Marking points are allocated based on typical exam expectations for a 5-mark question.

QUESTION 4 (5 marks)

Source: CSC429-2023-02, page [2]

Unofficial model answer:

a) Von Neumann input and output components from the Jacquard Loom excerpt (≈3 marks). In the Von Neumann model the *input* component supplies data/instructions to the machine and the *output* component presents results. In the excerpt, the **punched cards** (with rows of punched holes encoding the textile design) act as the **input** device: the hole pattern is the stored program/data read into the loom, analogous to a program on cards being fed to a stored-program computer. The **woven textile/fabric produced by the loom** (the patterned cloth that results from following the card sequence) is the **output**: it is the processed result delivered by the machine. The loom mechanism itself corresponds to the processing/control unit that interprets the input cards and drives the weaving, but the question asks only for input and output, so the punched cards = input and the woven patterned cloth = output.

b) Similarity and dissimilarity: mechanical calculating machine vs. Atanasoff–Berry Computer (≈2 marks). *Similarity:* both are **digital** machines that perform arithmetic/computation automatically from a defined sequence of operations rather than by continuous physical measurement — i.e., both automate calculation and were early steps toward programmable computing. *Dissimilarity:* the mechanical calculating machine (e.g., Pascal/Leibniz type) is **mechanical**, using gears and wheels with no electronic storage or binary logic, whereas the **Atanasoff–Berry Computer (ABC)** was **electronic**, using vacuum tubes and binary representation with regenerative capacitor memory — it was the first electronic digital computer, so the two differ in implementation technology (mechanical vs. electronic) and in number representation/memory.

Marking points:

• 1 mark: identifies punched cards as the input component (data/program supplied to the loom)

• 1 mark: identifies the woven textile/patterned cloth as the output component (result produced)

• 1 mark: correctly relates input/output to the Von Neumann stored-program/data-flow idea (cards = program/data in, cloth = result out)

• 1 mark: states ONE valid similarity between the mechanical calculating machine and the ABC (e.g., both are digital/automatic calculating machines)

• 1 mark: states ONE valid dissimilarity (e.g., mechanical gears vs. electronic vacuum-tube binary machine with regenerative memory)

Model confidence: medium.

Model note: Figure 1 is supplied only as a caption/excerpt of the Wikipedia Loom article; the answer is based on the extracted text, not the image. Part (b) references a comparison presumably covered in lecture material on early computing devices; the specific machines intended (e.g., Pascal's Pascaline vs. ABC) are assumed. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 5 (10 marks)

Source: CSC429-2017-07, page [4]

Unofficial model answer:

a) A replacement algorithm is needed in a cache because the cache is much smaller than main memory, so when a cache miss occurs and the requested block must be brought in, the cache may already be full. The replacement algorithm decides which existing cache block (line) to evict to make room for the new block. A

© Hak Cipta Universiti Teknologi MARA

good replacement policy aims to evict blocks that are least likely to be used again, thereby maximising the hit ratio and minimising unnecessary memory traffic. Without a replacement algorithm, the cache could not handle capacity or conflict misses in a set-associative or fully associative organisation.

b) The main replacement algorithms are: (1) Random replacement — a randomly chosen block is evicted; simple to implement in hardware but gives no regard to usage history. (2) FIFO (First-In, First-Out) — the block that has been in the cache the longest is replaced; easy to implement with a queue or counter, but may evict a frequently used block. (3) LRU (Least Recently Used) — the block that has not been referenced for the longest time is replaced; exploits temporal locality and generally gives good performance, but true LRU is costly to implement for high associativity because it requires tracking the order of all accesses. (4) LFU (Least Frequently Used) — the block with the fewest references is replaced; favours frequently used blocks but can retain stale blocks that were heavily used in the past. (5) Optimal (MIN) replacement — replaces the block that will not be used for the longest time in the future; it is a theoretical benchmark that cannot be implemented in practice because future references are unknown. Other variants include pseudo-LRU (approximates LRU with less hardware) and clock/second-chance algorithms, which are practical approximations of LRU.

Marking points:

• 1 mark: state that cache is smaller than main memory and replacement is needed on a miss when the target set/line is full

• 1 mark: explain that the algorithm selects a victim block to evict to make space for the new block

• 1 mark: mention goal of maximising hit ratio / minimising unnecessary evictions of useful blocks

• 1 mark: name Random replacement and briefly describe it

• 1 mark: name FIFO and briefly describe it

• 1 mark: name LRU and briefly describe it

• 1 mark: name LFU and briefly describe it

• 1 mark: name Optimal (MIN) and note it is theoretical / not implementable

• 1 mark: mention at least one practical approximation such as pseudo-LRU or clock/second-chance

• 1 mark: overall clarity, correct terminology, and coverage of all types

Model confidence: high.

Model note: The question text does not specify a particular cache organisation; the answer assumes set-associative or fully associative caches where replacement is relevant. The list of replacement types is based on standard computer-organization textbooks; some courses may emphasise a subset (e.g., Random, FIFO, LRU, Optimal) and treat LFU as optional. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 6 (10 marks)

Source: CSC429-2024-07, page [4]

Unofficial model answer:

a) The four classifications of parallel processors (Flynn's taxonomy) are: (1) SISD — Single Instruction, Single Data: a conventional uniprocessor such as a standard PC executing one instruction stream on one data stream. (2) SIMD — Single Instruction, Multiple Data: one control unit drives many processing elements, e.g., array processors, vector processors, and GPU/SIMD extensions (MMX/SSE/AVX). (3) MISD — Multiple Instruction, Single Data: multiple instruction streams operate on the same data stream; it is rarely used in practice, with examples such as systolic arrays or fault-tolerant systems with replicated processing. (4) MIMD — Multiple Instruction, Multiple Data: multiple processors execute independent instruction streams on independent data, e.g., multicore processors, SMP servers, and clusters/distributed systems. b) One advantage of cluster computer architecture is high availability/fault tolerance: because a cluster consists of multiple independent computers connected by a network, if one node fails the remaining nodes can continue providing service (failover), so the system as a whole is more reliable than a single large machine. A second acceptable advantage is scalability/cost-effectiveness: commodity nodes can be added incrementally to increase performance at lower cost than a single high-end supercomputer.

© Hak Cipta Universiti Teknologi MARA

Marking points:

• 1 mark: SISD correctly named with a valid example (e.g., uniprocessor/PC).

• 1 mark: SIMD correctly named with a valid example (e.g., vector/array processor, GPU, SIMD extensions).

• 1 mark: MISD correctly named with a valid example (e.g., systolic array, replicated fault-tolerant system).

• 1 mark: MIMD correctly named with a valid example (e.g., multicore, SMP, cluster).

• 2 marks: all four classifications correctly named (0.5 each) — total for part (a) = 6 marks.

• 2 marks: one clear advantage of cluster architecture stated (e.g., high availability/fault tolerance or scalability/cost-effectiveness).

• 2 marks: advantage explained with justification (e.g., node failure does not stop service; commodity nodes can be added incrementally).

• Total: 10 marks.

Model confidence: high.

Model note: Assumes the question refers to Flynn's taxonomy, the standard classification taught for parallel processors. If the course used a different taxonomy (e.g., shared-memory vs distributed-memory), the named categories would differ; the marking scheme should be adjusted accordingly. No figure or table was supplied.

QUESTION 7 (10 marks)

Source: CSC429-2026-02, page [3]

Unofficial model answer:

a) The machine cycle (also called the instruction cycle) is the sequence of operations a CPU performs to execute one instruction. It comprises four phases: Fetch, Decode, Execute, and Store (or Write-back). In the Fetch phase, the Program Counter (PC) holds the address of the next instruction; the Control Unit (CU) places this address on the address bus and issues a memory read signal. The Main Memory returns the instruction word on the data bus, which is loaded into the Instruction Register (IR). The PC is then incremented to point to the next instruction. In the Decode phase, the CU decodes the opcode in the IR to determine the operation and identifies the operands, possibly using the Arithmetic Logic Unit (ALU) for address calculation. In the Execute phase, the CU asserts control signals to the relevant functional units: the ALU performs arithmetic/logic operations, registers supply operands, and the CU coordinates data movement. In the Store phase, the result is written back to a register or to main memory, and status flags (e.g., zero, carry) in the Status/Flag Register are updated. The CU sequences all phases using the system clock, while registers (PC, IR, MAR, MDR, general-purpose registers) and buses (address, data, control) provide the data path and communication. Thus, the machine cycle is a coordinated operation of the Control Unit, ALU, registers, memory, and buses.

b) If main memory is corrupted, the machine cycle is disrupted at multiple points. During Fetch, a corrupted memory location may return an invalid instruction word or a parity/ECC error. The IR may load garbage, and the CU may decode an illegal opcode, causing an invalid instruction exception or a halt. If the PC or stack contents in memory are corrupted, the fetch address may point to an invalid location, leading to a page fault or segmentation fault. During Execute, operands fetched from corrupted memory may be incorrect, producing wrong results or triggering arithmetic exceptions (e.g., divide by zero if a divisor is corrupted). During Store, writing to a corrupted memory cell may fail or corrupt adjacent data, and error-detection circuitry (parity/ECC) may raise a machine check or memory error interrupt. The CU may enter an error-handling routine or halt the system. In summary, memory corruption can cause data errors, illegal instruction traps, address faults, and system instability, preventing the machine cycle from completing correctly.

Marking points:

• 1 mark: Correct identification of the four phases of the machine cycle (Fetch, Decode, Execute, Store/Write-back).

• 1 mark: Explanation of Fetch phase with PC, MAR, MDR, IR, and memory read.

• 1 mark: Explanation of Decode phase with Control Unit and instruction decoding.

• 1 mark: Explanation of Execute phase with ALU and control signals.

© Hak Cipta Universiti Teknologi MARA

• 1 mark: Explanation of Store phase with write-back and flag update.

• 1 mark: Identification of responsible components: CU, ALU, registers (PC, IR, MAR, MDR), memory, and buses.

• 1 mark: Description of effect on Fetch (invalid instruction, illegal opcode, exception).

• 1 mark: Description of effect on Decode/Execute (wrong operands, arithmetic exceptions).

• 1 mark: Description of effect on Store (write failure, error detection, interrupt).

• 1 mark: Overall consequence: system halt, error-handling routine, or instability.

Model confidence: high.

Model note: This is an unofficial model answer, not an official UTM marking scheme. The question does not refer to a specific figure or table; the answer assumes a standard von Neumann machine cycle. Marks are allocated evenly across subparts (5 marks each) with sub-marking as listed.

QUESTION 8 (10 marks)

Source: CSC429-2023-02, page [3]

Unofficial model answer:

Subpart (a) — Convert 43.046875_{10} to IEEE 754 single precision (hex): The number is positive, so the sign bit S = 0. Split into integer and fraction: 43 = 101011_2 (32+8+2+1). The fraction 0.046875 = 3/64 = 0.000011_2 (since 1/32 + 1/64 = 0.03125 + 0.015625). So 43.046875 = 101011.000011_2. Normalise: $1.01011000011_2 \times 2^5$. The biased exponent = 5 + 127 = 132 = 10000100_2. The mantissa (fraction after the leading 1) is 01011000011 followed by zeros to 23 bits: 01011000011000000000000. Assemble: S=0, E=10000100, M=01011000011000000000000 → 0 10000100 01011000011000000000000. Group into hex nibbles: 0100 0010 0010 1100 0011 0000 0000 0000 = 0x422C3000. (Check: 0x422C3000 ≈ 43.046875.)

Subpart (b) — Convert 0xC4780000 (IEEE single precision) to decimal: Write the 32 bits: 1100 0100 0111 1000 0000 0000 0000 0000. Sign bit S = 1 → negative. Exponent field = 10001000_2 = 136, so unbiased exponent = 136 − 127 = 9. Mantissa field = 1111000…0, so significand = 1.1111_2 = 1 + 1/2 + 1/4 + 1/8 + 1/16 = 1.9375. Value = $-1.9375 \times 2^9 = -1.9375 \times 512 = -992.0$. Hence 0xC4780000 = -992.0_{10}.

Marking points:

• (a) 1 mark: correct sign bit S = 0 (positive number).

• (a) 1 mark: correct binary of integer part 43 = 101011_2.

• (a) 1 mark: correct binary of fraction 0.046875 = 0.000011_2.

• (a) 1 mark: correct normalisation $1.01011000011_2 \times 2^5$ and biased exponent 132 = 10000100_2.

• (a) 1 mark: correct 23-bit mantissa field and final hex 0x422C3000 (allow carry-through of earlier error).

• (b) 1 mark: correct sign interpretation S = 1 → negative.

• (b) 1 mark: correct exponent field 10001000_2 = 136 and unbiased exponent 9.

• (b) 1 mark: correct significand 1.1111_2 = 1.9375.

• (b) 1 mark: correct computation $-1.9375 \times 2^9 = -992.0$.

• (b) 1 mark: final decimal answer stated as −992.0 (with correct sign and value).

Model confidence: high.

Model note: The question text contains OCR artefacts: '43.046875io' is read as 43.046875 decimal, and 'C478 OOOOh' is read as 0xC4780000 (the 'OOOO' being zeros). If the intended hex were different (e.g. C4780000 vs C4780001), the decimal result would change slightly; the answer assumes the standard reading 0xC4780000. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 9 (10 marks)

Source: CSC429-2019-06, page [3]

Unofficial model answer:

a) Registers are small, high-speed storage locations inside the CPU used to hold data, instructions, addresses, or status information temporarily during processing. Two types of registers are: (1) Program

Counter (PC) – holds the address of the next instruction to be fetched; (2) Instruction Register (IR) – holds the instruction currently being executed. Other acceptable examples include Memory Address Register (MAR), Memory Data Register (MDR), Accumulator (AC), and Status/Flag Register.

b) Fetch operation: The CPU reads the next instruction from memory. The address of that instruction is taken from the Program Counter (PC) and placed in the Memory Address Register (MAR). The instruction is then read from main memory into the Memory Data Register (MDR) and copied to the Instruction Register (IR). The PC is incremented to point to the next instruction. Decode operation: The control unit examines the opcode in the Instruction Register (IR) to determine what operation is required and generates the appropriate control signals to execute it.

c) A dedicated bus has separate, physically distinct lines for each function (e.g., separate address and data lines), so address and data can be transferred simultaneously, giving higher performance but requiring more lines and cost. A multiplexed bus shares the same physical lines for different functions (e.g., address and data are time-multiplexed), which reduces the number of lines and cost but requires time-sharing, so transfers cannot occur simultaneously and performance is lower.

Marking points:

• 1 mark: Correct definition of registers as small, fast CPU storage for temporary data/instructions/addresses.

• 1 mark: One correct type of register (e.g., PC, IR, MAR, MDR, Accumulator).

• 1 mark: Second correct type of register.

• 1 mark: Fetch operation correctly described (PC provides address, instruction fetched from memory into IR, PC incremented).

• 1 mark: Decode operation correctly described (control unit interprets opcode in IR and generates control signals).

• 1 mark: Clear distinction that dedicated bus has separate lines for different functions.

• 1 mark: Clear distinction that multiplexed bus shares lines for different functions (time-multiplexed).

• 1 mark: Correct implication for dedicated bus (simultaneous transfer, higher performance/cost).

• 1 mark: Correct implication for multiplexed bus (reduced lines/cost, lower performance/time-sharing).

• 1 mark: Overall clarity and use of correct terminology across all parts.

Model confidence: high.

Model note: This is an unofficial model answer, not an official UTM marking scheme. The question does not refer to any figure or table, so no limitation applies.

QUESTION 10 (10 marks)

Source: CSC429-2016-06, page [5]

Unofficial model answer:

a) Direct mapping does not need a replacement algorithm. In direct mapping each main-memory block maps to exactly one cache line (line = block number mod number of lines), so when a new block is brought in there is only one possible line it can occupy; that line is simply overwritten. No choice among candidate lines arises, hence no replacement policy is required. (Fully associative and set-associative caches do require a replacement algorithm because a new block may be placed in any line of the cache or of its set.)

b) Four replacement algorithms:

1. Random replacement: a line is chosen at random from the candidate lines (the whole cache for fully associative, or the target set for set-associative). It is simple and cheap in hardware, requires no bookkeeping, but gives no locality-based performance guarantee.
2. FIFO (First-In, First-Out): the line that has been resident in the cache the longest is replaced. It is implemented with a circular queue/age counter per line; it is simple but can evict a heavily used line simply because it is old.
3. LRU (Least Recently Used): the line that has not been referenced for the longest time is replaced, exploiting temporal locality. It gives good hit ratios but needs recency bookkeeping (e.g., counters or a reference/use bit

© Hak Cipta Universiti Teknologi MARA

matrix), which is costly for high associativity.

4. LFU (Least Frequently Used): the line with the smallest number of references (access count) is replaced. It favours frequently used blocks but suffers from stale counts and does not adapt quickly to changing working sets; counters must be maintained per line.

(Other acceptable algorithms include Optimal/MIN, NRU, and Second-Chance/Clock; any four correctly described policies earn full credit.)

Marking points:

• *1 mark: (a) correctly identifies direct mapping as the mapping function that needs no replacement algorithm.*

• *1 mark: (a) gives a valid reason (each block maps to exactly one line, so the line is simply overwritten).*

• *1 mark: (b) names and correctly describes Random replacement.*

• *1 mark: (b) names and correctly describes FIFO replacement.*

• *1 mark: (b) names and correctly describes LRU replacement.*

• *1 mark: (b) names and correctly describes LFU replacement (or another valid algorithm such as Optimal, NRU, Second-Chance).*

• *2 marks: overall accuracy/terminology and completeness across the four algorithms (e.g., each description states the selection rule and a brief property).*

• *2 marks: clarity and structure of the answer, with each algorithm clearly separated and no conceptual errors (e.g., confusing mapping with replacement).*

Model confidence: high.

Model note: This is an unofficial model answer, not an official UTM marking scheme. The question text does not include a figure or table, so no figure-based limitation applies. Mark allocation is inferred because the paper only states the total (10 marks); the split between (a) and (b) and the weighting of descriptions are assumed.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION 11 (20 marks)

Source: CSC429-2026-02, page [6]

Unofficial model answer:

Interpretation of Figure 5: The expression is read as z = A + (C²/B)·D − 3A, i.e. z = A + ((C*C)/B)*D − 3*A. Registers A, B, C, D are fixed (read-only operands); x, y, z are scratch; the final result must end in z. Note: the OCR text 'z-A+c2/B*D-3A' is ambiguous, so the expression above is the assumed reading (see notes).

(a) Zero-address (stack) program. All operands must be pushed onto the stack; results are left on the stack and finally popped into z. Using the stack, compute C*C, divide by B, multiply by D, then add A and subtract 3*A:

```
PUSH C
PUSH C
MUL
PUSH B
DIV
PUSH D
MUL
PUSH A
ADD
PUSH A
PUSH A
ADD
PUSH A
ADD
SUB
POP z
```

(Equivalently, the 3A term can be formed as PUSH A / PUSH A / ADD / PUSH A / ADD, then SUB.) The stack holds the running value; the last POP z stores the final answer in z.

(b) One-address program. One operand is the accumulator (implicit) and the other is a named register; results go to the accumulator. Use x and y as temporaries:

```
LOAD C
MUL C
STOR x
LOAD x
DIV B
STOR x
LOAD x
MUL D
STOR x
LOAD A
ADD x
STOR y
LOAD A
MUL A
ADD A
ADD A
```

© Hak Cipta Universiti Teknologi MARA

STOR x
LOAD y
SUB x
STOR z

Here x holds the C²/B·D term, y holds A + that term, and x is reused to hold 3A; the final SUB leaves the answer in the accumulator, which is stored to z.

(c) Three-address program. Each instruction names two sources and one destination, so the expression is built directly with temporaries x and y:

MUL (C, C, x)
DIV (x, B, x)
MUL (x, D, x)
ADD (A, x, y)
MUL (A, A, x)
ADD (x, A, x)
ADD (x, A, x)
SUB (y, x, z)

This computes x = C²/B·D, y = A + x, x = 3A, and finally z = y − x = A + C²/B·D − 3A, with the answer in z as required.

Marking points:

• *2 marks: correct interpretation of Figure 5 as z = A + (C*C)/B*D - 3*A (or clearly stated equivalent)*

• *6 marks: zero-address program — correct PUSH/POP usage, correct operator order (MUL, DIV, MUL, ADD, SUB), final POP z (2 marks per logical stage)*

• *6 marks: one-address program — correct LOAD/STOR with accumulator, correct use of temporaries x and y, correct final STOR z*

• *4 marks: three-address program — correct ADD/SUB/MUL/DIV (p,q,r) form, correct temporaries, final result in z*

• *2 marks: overall correctness — final answer stored in z in all three versions and fixed registers A, B, C, D never overwritten*

Model confidence: medium.

*Model note: Figure 5 is supplied only as OCR text 'z-A+c2/B*D-3A'; the expression was interpreted as z = A + C^2/B*D - 3A. If the intended expression differs (e.g. z = A + C^2/(B*D) - 3A or z = A + C^2/B*D - 3*A with different grouping), the instruction sequences must be re-ordered accordingly. Table 1 gives only the instruction mnemonics; operand conventions (e.g. whether one-address ADD p means ACC <- ACC + p) are assumed standard. This is an unofficial model answer, not an official UTM marking scheme.*

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA